

Homework 05

Set a random seed before each simulation question and report it. You may use standard packages unless a question asks you to write the calculation yourself. Use equal neighbor weights throughout. Here p counts the covariates used to measure distance; do not add an intercept column to a KNN predictor matrix. R and Python use different random-number generators, so their exact simulation results need not agree.

Question 1: Estimate the bias and variance of KNN

In our previous homework, we used repeated simulations to study the behavior of linear regression estimators. We will now use the same idea for KNN. Consider the regression model

$$Y_i = f(\mathbf{X}_i) + \epsilon_i, \quad f(\mathbf{x}) = 0.5x_1 + \sin(x_2) - 0.3x_3^2,$$

where the three covariates are independent standard normal variables and $\epsilon_i \sim \mathcal{N}(0, 0.1^2)$ independently of the covariates and other observations. Generate $n = 400$ training observations. Our goal is to estimate the mean response at

$$\mathbf{x}_0 = (0.5, 0.7, 1)^T.$$

Use Euclidean distance and consider $k = 1, 3, 5, \dots, 29$.

- a. Calculate $f(\mathbf{x}_0)$. Generate one training dataset and obtain the KNN prediction at $\mathbf{x}_0$ for each value of k .
- b. Independently repeat the generation of the training covariates and responses 200 times. Within each repetition, use the same training data for every value of k . Store the predictions in a matrix with 200 rows and one column for each value of k . Use this matrix to estimate the bias, variance, and mean squared error of the estimated mean response. Use divisor 200 when calculating the empirical variance, and verify numerically that

$$\text{mean squared error} = \text{squared bias} + \text{variance}.$$

Plot squared bias, variance, and mean squared error against k in one figure. Explain the pattern and compare it with the bias-variance trade-off for ridge regression or lasso.

ANSWER: **a.** The target is the mean response, whose true value is

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.594218.$$

We first generate one training dataset and average the responses of the nearest k observations for each choice of k .

► Show the workflow

	k	prediction	truth
1	1	0.7888302	0.5942177
2	3	0.6621674	0.5942177
3	5	0.6642853	0.5942177
4	7	0.6100117	0.5942177
5	9	0.6318671	0.5942177
6	11	0.4925114	0.5942177
7	13	0.5222989	0.5942177
8	15	0.5810203	0.5942177
9	17	0.5738171	0.5942177
10	19	0.5751479	0.5942177
11	21	0.5800517	0.5942177
12	23	0.5519177	0.5942177
13	25	0.5214556	0.5942177
14	27	0.4952957	0.5942177
15	29	0.4972667	0.5942177

- b.** Each column of the prediction matrix contains 200 estimates of the same target. Its mean minus $f(\mathbf{x}_0)$ estimates bias; its variance, calculated with divisor 200, estimates sampling variance. Averaging the squared differences from $f(\mathbf{x}_0)$ estimates MSE. Using the same divisor makes the empirical decomposition an exact identity, apart from rounding.

► Show the workflow

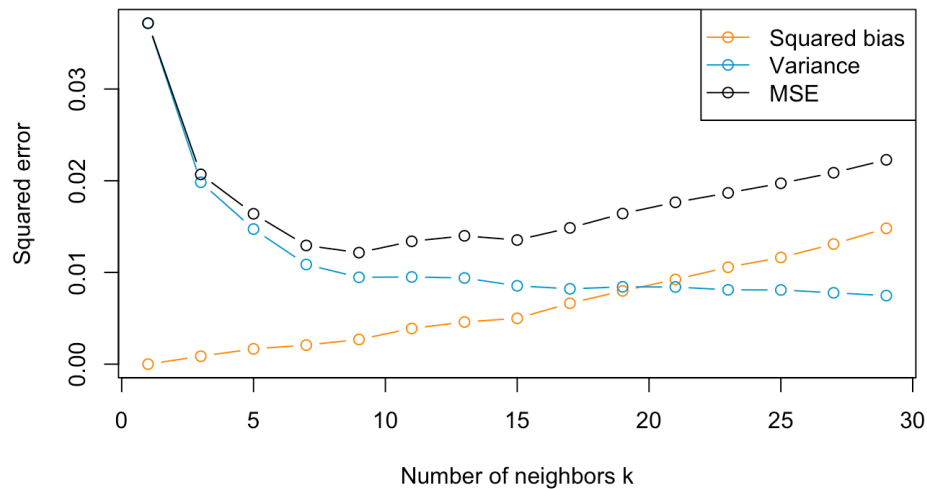
	k	bias	variance	MSE
1	1	-0.001248923	0.037181204	0.03718276
2	3	-0.029361680	0.019831580	0.02069369
3	5	-0.040815728	0.014731018	0.01639694
4	7	-0.045589732	0.010865566	0.01294399
5	9	-0.051867114	0.009467440	0.01215764
6	11	-0.062360901	0.009506180	0.01339506
7	13	-0.067797856	0.009397702	0.01399425
8	15	-0.070662763	0.008542311	0.01353554
9	17	-0.081499864	0.008213727	0.01485595
10	19	-0.089385641	0.008429774	0.01641957
11	21	-0.096080776	0.008413451	0.01764497
12	23	-0.102803643	0.008103017	0.01867161
13	25	-0.107847900	0.008091147	0.01972232

```
14 27 -0.114428351 0.007777947 0.02087180
15 29 -0.121688194 0.007474576 0.02228259
```

► Show the workflow

[1] 6.765422e-17

► Show the workflow

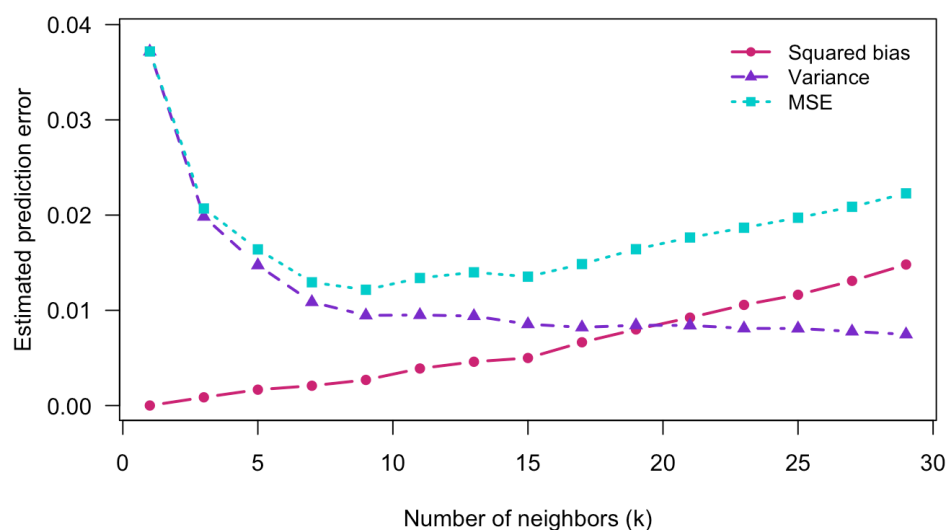


Larger k averages more responses, reducing variability, but it also includes observations farther from the target. Their mean responses can differ more from $f(x_0)$, introducing bias. Here an intermediate neighborhood gives a lower MSE than either very few or many neighbors. Small irregularities in the curves are expected with 200 repetitions; monotonic changes in each component are not guaranteed.

Applying the `plot-formatting` skill gives the plot a descriptive title, clearer labels, pink, purple, and cyan colors, distinct point shapes and line types, and a larger legend.

► Show the workflow

Bias-variance trade-off for KNN



I prefer the revised plot because the title states the main comparison, the axes are more descriptive, and the curves remain distinguishable through line types and point shapes as well as color. The larger margins and expanded y-axis limit reduce the risk of clipped labels or points. The plotted values and interpretation are unchanged.

Increasing k plays a similar smoothing role to increasing the penalty in ridge regression or lasso: predictions become less flexible, with a trade-off between bias and variance. KNN changes which local responses are averaged; ridge and lasso shrink fitted coefficients. In this experiment both the training locations and their responses change across repetitions, so the variance includes changing neighborhoods as well as response noise. It is therefore not simply $0.1^2/k$, which describes the conditional noise variance when the neighbor locations are fixed.

Question 2: Compare KNN with lasso

Use the response model from Question 1, including the noise standard deviation 0.1, but generate $p = 30$ covariates. The response still depends only on the first three. Consider these two settings:

- **Setting 1:** all 30 covariates are independent standard normal variables.
- **Setting 2:** the covariates have a multivariate normal distribution with mean zero and covariance matrix Σ , where

$$\Sigma_{jj} = 1, \quad \Sigma_{jl} = 0.8 \quad \text{for } j \neq l.$$

In each setting, generate 400 training observations and 1,000 independent test observations. Fit 5NN using Euclidean distance on the supplied covariate scale. All covariates already have population variance one.

For lasso, use a linear model containing the 30 covariates and a separately fitted, unpenalized intercept. Follow the Week 4 objective,

$$L_{\lambda}(\beta) = \frac{1}{2n} \| \bar{y} - X\beta \|_2^2 + \lambda \|\beta\|_1,$$

where $X \in \mathbb{R}^{n \times 30}$ contains the centered and standardized covariates and $\bar{y}$ is the centered training response. Select the penalty by ten-fold cross-validation over 41 logarithmically spaced values from 10^{-4} to 1. Estimate all means and scales using only the training portion of each fold, with divisor equal to that portion's sample size. After selecting the penalty, refit on all training observations and apply the fitted transformations unchanged to the test observations. Do not add nonlinear terms to the lasso model.

- a. Fit both methods in each setting. Use the same training and test data for the two methods within a setting. Report their test MSE values in one table, together with the selected lasso penalty and the number of nonzero lasso slopes. You only need to generate one dataset for each setting; repeated simulations are not required.
- b. Compare the results between the two settings. Explain how correlation can change the usefulness of Euclidean neighborhoods, and why lasso does not use all covariates in the same way as KNN.
- c. The response model contains $\sin(X_2)$ and X_3^2 , but the lasso fit uses only linear terms. Explain how this difference affects the comparison. Would the results justify a general claim that correlation always helps KNN, or that one method is always better? Explain why.

ANSWER: **a.** We compare the two methods on the same observations within each setting. KNN uses all 30 covariates when finding neighbors. Lasso can set slopes to zero, but it is restricted to a linear prediction rule. The following code selects the lasso penalty using training data only.

► Show the workflow

Loading required package: Matrix

Loaded glmnet 5.0

► Show the workflow

Setting	KNN_MSE	Lasso_MSE	Lambda	Nonzero
Independent	0.5565	0.2602	0.0251	11
Correlated	0.1609	0.2259	0.0100	16

The scales use divisor equal to the number of observations being fitted. Centering the response and adding its mean back gives an unpenalized intercept. Here `glmnet`'s `lambda` and `scikit-learn`'s `alpha` both use the penalty in the stated $1/(2n)$ objective; `glmnet`'s `alpha = 1` requests lasso. R and Python generate different observations, so their numerical results need not match.

b. In these realizations, lasso has the smaller test MSE with independent covariates, whereas KNN has the smaller test MSE with correlated covariates. With independent covariates, many coordinates that do not affect the response contribute to Euclidean distance. The nearest observations in 30-dimensional Euclidean distance need not be the nearest in the three relevant coordinates.

With strong positive correlation, the covariates share substantial variation. Euclidean neighborhoods can then be more informative about the response-relevant coordinates. Lasso treats covariates differently: it estimates a separate slope for each and can set some slopes to zero. In the correlated setting, a covariate outside the first three can also act as a proxy for a relevant covariate, and several correlated covariates can compete in the fitted model.

c. Lasso uses X_2 and X_3 as linear terms, so it cannot reproduce $\sin(X_2)$ or the quadratic contribution $-0.3X_3^2$. Its intercept can account for the average quadratic contribution, but not its variation across observations. KNN does not impose a linear form, although its accuracy depends on whether it finds informative neighbors. The comparison therefore reflects both neighborhood quality and the restrictions of the fitted lasso model. Two single datasets do not show that correlation always helps KNN or that either method is universally better.

Question 3: Observed dimension and latent dimension

The lecture's handwritten digit example shows that KNN can work with many measured covariates. We will use a simulation to investigate why the structure of those covariates matters.

Let $p = 100$ be the number of observed covariates and let m be the number of latent variables. For each observation, generate

$$X_i = AZ_i + \eta_i, \quad Y_i = Z_{i1} + Z_{i2} + \epsilon_i,$$

where $A \in \mathbb{R}^{100 \times m}$ has independent entries from $\text{Uniform}[-2, 2]$. The entries of $Z_i \in \mathbb{R}^m$ and $\eta_i \in \mathbb{R}^{100}$, and the scalar ϵ_i , are independent standard normal variables, also independent of A . Generate A once and use it for all training and test observations in that dataset.

Compare $m = 3$ and $m = 30$. In each setting, generate 200 training observations and 200 independent test observations. Fit KNN using only the 100 observed covariates in X , with Euclidean distance on their generated scale. The latent variables in Z are used to generate the data but are not available to the fitted model. Consider $k = 2, 10, 18, \dots, 82$.

- a. Generate one dataset for each value of m . Fit KNN for every value of k and calculate test MSE. Within a setting, use the same training and test observations for all values of k .
- b. Repeat the complete experiment 50 times, generating a new A and new observations in each repetition. Plot average test MSE against k , with one curve for each value of m . These curves compare the specified choices of k in a controlled simulation; they are not a cross-validation procedure for selecting a final model.
- c. Both settings have 100 observed covariates, and the response depends on only two latent variables. Explain why their KNN performance can still differ. What role does the noise η_i play, and why are the observed data not confined exactly to an m -dimensional linear space?

ANSWER: **a.** The observed dimension is 100 in both settings. We generate the training and test observations using the same matrix A , then use only X to fit KNN.

► Show the workflow

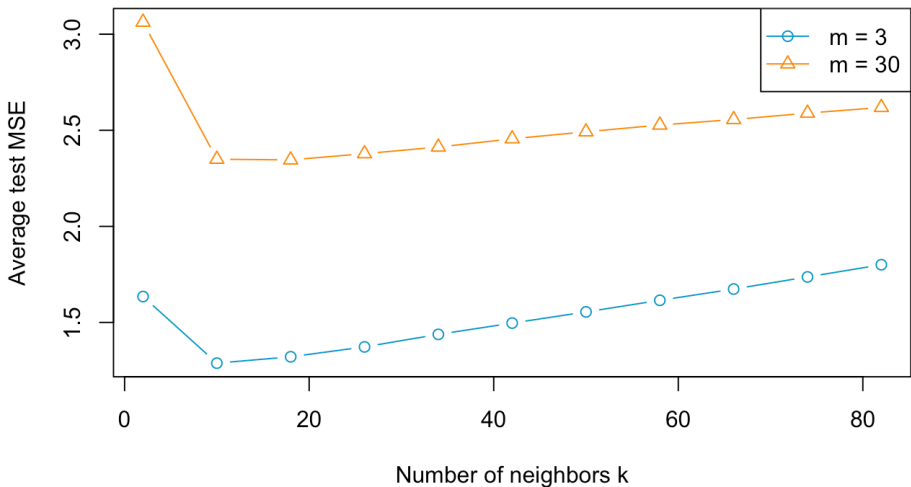
	k	m3	m30
1	2	1.449265	3.457456
2	10	1.253345	2.583372
3	18	1.292923	2.533071
4	26	1.379955	2.524038
5	34	1.448416	2.512187
6	42	1.505892	2.560783
7	50	1.590057	2.562911
8	58	1.660186	2.593970
9	66	1.739597	2.607162
10	74	1.816963	2.651181
11	82	1.871802	2.672724

b. The following loop repeats the entire experiment, including generation of A . The result array stores repetitions, choices of k , and the two latent dimensions along its three axes.

► Show the workflow

	k	m3	m30
1	2	1.634822	3.062662
2	10	1.288062	2.349524
3	18	1.321139	2.345840
4	26	1.372628	2.378219
5	34	1.437850	2.412709
6	42	1.496497	2.455664
7	50	1.554715	2.492101
8	58	1.615062	2.527212
9	66	1.673644	2.556039
10	74	1.736733	2.589309
11	82	1.800274	2.618853

► Show the workflow



The $m = 3$ curve has lower average test MSE in this experiment. Averaging a few neighbors reduces noise, while very large neighborhoods blur differences in the regression function. With $m = 30$, even a small neighborhood is less local in the latent variables. The two curves need not have their

minima at the same k . These are simulation averages over independent experiments, not validation scores used to choose a model for a final test set.

c. When $m = 3$, most of the structured variation in 100 covariates is driven by three latent variables. With $m = 30$, distance also responds to many latent directions that do not enter the response. Two observations close in the two relevant latent coordinates may be far apart in those other directions, making Euclidean neighborhoods less useful for predicting Y .

The measurement noise η_i perturbs every observed coordinate and can change which observations are nearest. Without it, AZ_i lies in the column space of A , of dimension at most m . With independent noise in all 100 coordinates, X_i is not confined to that space. Thus the simulation concerns a lower-dimensional signal with noise, not data lying exactly in an m -dimensional subspace. Keeping the response dependent on two latent variables does not make distances in all observed coordinates equally informative in the two settings.

Question 4: Classify handwritten digits

Use `zip.train` and `zip.test` from the R package `ElemStatLearn`, as in the lecture. The package has been retired from the active CRAN repository. If needed, install the [archived package](#) into your usual R library, then load the two datasets. Python users can use the course copies of the same [training data](#) and [test data](#). These compressed CSV files have no headers and are included in the homework ZIP under `data/knn`. Run the Python code from the folder containing `data`.

Each image has $16 \times 16 = 256$ pixel values. The first column is the digit label and the remaining columns are the pixel values. Use the first 1,000 rows of `zip.train` and the first 500 rows of `zip.test`. Use the original pixel values without standardization, and do not include the digit label in a distance calculation.

For a test image x_0 and a training image x_i , the two distances are

$$d_E(x_0, x_i) = \sqrt{\sum_{j=1}^{256} (x_{0j} - x_{ij})^2}, \quad d_M(x_0, x_i) = \sum_{j=1}^{256} |x_{0j} - x_{ij}|.$$

- a. Write a 5NN classification function that allows either Euclidean or Manhattan distance. For each test image, calculate its distances to the training images directly, find the five nearest, and return the most frequent digit label. Break distance ties by the smaller training row number and voting ties by the smallest digit. Do not use a built-in distance or KNN function for this part.
- b. Apply the classifier to all 500 test images using each distance. For each fit, report the test classification error and a 10×10 confusion matrix, with rows representing predicted digits and columns representing true digits. Keep all digit labels from 0 through 9 in the matrix, including any with a zero count. Which digits are most often confused with one another?
- c. For Euclidean distance, display three incorrectly classified test images, together with their true and predicted labels. Comment on whether the mistakes are visually understandable. Compare the overall results of the two distance measures. Can they have similar total error while making mistakes on different images? Use your predictions to support your answer.

ANSWER: With the specified training and test rows, Euclidean 5NN makes 61 errors out of 500, and Manhattan 5NN makes 69. Both recognize most digits, but some of their errors occur on different images.

a. **Write the classifier.** For each test image, calculate the distances, sort the training rows, and count the labels of the five closest images. Passing either of the distance functions below changes the distance without changing the remaining classification rule.

► Show the workflow

b. **Evaluate classification.** Each column of `y_hat` contains 500 predictions. The confusion matrices below have predicted digits in rows and true digits in columns.

► Show the workflow

Euclidean test error: 0.122

True											
Predicted	0	1	2	3	4	5	6	7	8	9	
0	118	0	7	1	0	4	2	0	2	0	
1	0	66	0	0	2	0	0	0	0	1	
2	0	0	39	1	0	0	2	0	0	1	
3	0	0	0	23	0	4	0	0	2	0	
4	0	0	1	0	27	1	1	1	0	1	
5	0	0	0	1	0	13	0	0	0	0	
6	0	2	0	0	0	1	41	0	2	0	
7	0	0	0	0	1	1	0	34	0	4	
8	0	0	3	2	0	2	0	0	37	0	
9	1	0	0	1	4	0	0	1	1	41	

Manhattan test error: 0.138

True											
Predicted	0	1	2	3	4	5	6	7	8	9	
0	118	0	6	1	0	5	3	0	4	0	
1	0	67	2	0	2	0	1	0	0	1	
2	0	0	35	0	0	0	2	0	0	0	
3	0	0	1	24	0	4	0	0	2	0	
4	0	0	1	1	27	1	0	1	0	1	
5	0	0	0	1	0	11	0	0	0	0	
6	0	1	0	0	0	2	40	0	2	0	
7	0	0	2	1	1	0	0	34	0	5	

8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

The test error is 0.122 for Euclidean distance and 0.138 for Manhattan distance. The most frequent individual confusion is a true 2 predicted as 0: seven images under Euclidean distance and six under Manhattan distance. Other frequent errors include 5 predicted as 0 or 3, 9 predicted as 7, and 4 predicted as 9. These are counts for this test set, whose digit classes have different sample sizes.

c. Inspect mistakes and compare predictions. We display the first three Euclidean errors, keeping the lecture's image orientation.

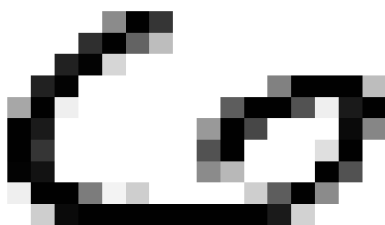
► Show the workflow

```
[1] "digit 6 taken"
```

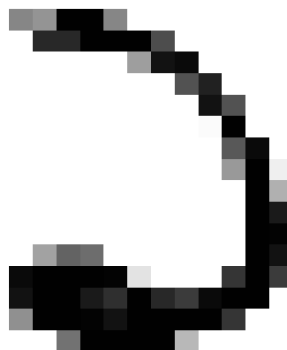
```
[1] "digit 2 taken"
```

```
[1] "digit 4 taken"
```

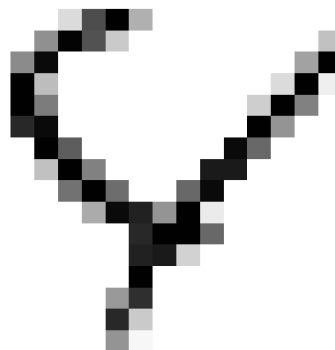
True: 6 Predicted: 0



True: 2 Predicted: 0



True: 4 Predicted: 7



► Show the workflow

Different predictions: 22

► Show the workflow

		Manhattan_correct	
Euclidean_correct		FALSE	TRUE
FALSE		58	3
TRUE		11	428

These images are test rows 4, 13, and 14, with true labels 6, 2, and 4, but predicted labels 0, 0, and 7. The 6 has a rounded lower loop, the 2 has an unusually curved outline, and the 4 lacks a typical horizontal crossbar. These shapes make pixel-distance mistakes plausible, even when a person can recognize the digit. Visual recognition also uses information about handwriting that this distance rule does not explicitly represent.

The methods disagree on 22 predicted labels. Euclidean distance is correct and Manhattan distance is wrong on 11 images; the reverse happens on three. On eight additional images, both are wrong but predict different digits. Thus, similar total errors can hide different mistakes. Euclidean distance has the smaller error on this split, but this comparison does not establish that it will always be better.

Question 5: Create a plot-formatting skill

Create a short `SKILL.md` file for improving the formatting of STAT 432 homework plots. You may use AI to help write it.

- Include a title, a description, and a few formatting rules. The description should limit the skill to STAT 432 homework plot improvements and require an explicit request to use it. Your rules should address margins, titles, axes, labels, colors, and sizes for plots produced in R or Python. Keep them portable: do not refer to specific homework questions.
- Ask your agent to install the skill. Explicitly call it to improve the plot from Question 1, then compare the original and revised versions. Do you like the changes? You may revise the skill and test it a couple more times, or uninstall it if it remains unhelpful.
- Append your final `SKILL.md` content to your submission and briefly explain your choices and experience.

plot-formatting skill

```
---
name: plot-formatting
description: Improve the formatting of STAT 432 homework plots when the student explicitly asks to use this skill.
---

# STAT 432 Plot Formatting

# Intent
```

Make STAT 432 homework plots easier to read without changing their data, calculations, or interpretation.

Instructions

- Preserve the plotted data and analytical meaning.
- Use clear outer margins and prevent titles, axis labels, legends, and tick labels from being clipped.
- Add a concise, informative title and descriptive x- and y-axis labels with units when applicable.
- Use readable pink, purple, and cyan colors with distinct line types or point shapes so the plot remains interpretable in grayscale.
- Choose readable font, point, line, and figure dimensions; use consistent sizing across related plots.
- Apply the same principles to plots produced in R or Python.
- Explain the formatting changes briefly and do not change substantive results.

The skill was installed and tested on the Question 1 bias-variance plot. I kept the data and calculations unchanged while improving the margins, title, axis labels, pink/purple/cyan colors, line types, point shapes, legend, and figure sizing. I prefer the revised version because it is clearer and the curves remain distinguishable without relying only on color.